SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I A.R. Afreen Fariha / 192571025 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

ANSWER BOOKLET – CSA17 Artificial Intelligence

Name: A.R. Afreen Fariha **Reg No:** 192571025

Assessment Tool 1 – Annexure A: Scenario Based Assignment (CO2)

Course outcome covered: CO2 – search and game-solving techniques (Greedy Search, A*, CSP, Minimax, Alpha-Beta pruning, heuristic functions).

Q.	Scenario	Techniques covered
1	AI drones delivering emergency medicine in a flood zone	Greedy Best-First Search, A*, heuristic accuracy, dynamic replanning
2	Smart-city traffic signal optimisation	Optimisation formulation, Hill Climbing, Simulated Annealing, Genetic Algorithms
3	Autonomous Mars rover in unknown terrain	Online search, LRTA*, D* Lite, partial observability, internal model
4	University exam timetabling	Constraint Satisfaction, backtracking, forward checking, AC-3
5	Strategic game AI for real-time decisions	Minimax, Alpha-Beta pruning, evaluation function, iterative deepening

Each answer follows the same structure: (1) understanding of the scenario, (2) identification of the key issues, (3) application of the relevant algorithm or concept with a worked example, (4) a justified decision, and (5) a short conclusion, matching the assessment rubric (understanding, issues, application, decision making and clarity).

Question 1: AI Drones for Emergency Medicine Delivery in a Flood Zone

1.0 Understanding the scenario and formal problem definition

A fleet of drones must carry emergency medicines across a flood-affected region. The region can be modelled as a weighted graph **G = (V, E)**. Each vertex is a waypoint (a depot, a relief camp, a hospital, a landing zone or a grid cell of airspace) and each edge is a flight leg between two waypoints. The environment has four properties that decide which search algorithm is appropriate:

- **Dynamic:** a leg may become unusable (flooded road corridor, collapsed bridge used as a landmark, no-fly zone created by rescue helicopters) while the drone is already flying.
- **Partially observable:** the drone only receives partial real-time updates, so its map of blocked legs is always incomplete and slightly out of date.
- **Stochastic / variable cost:** the cost of a leg is not its length. Headwind, rain, battery drain and terrain change the energy and time needed, so *g(n)* is a time-varying quantity.
- **Time and safety critical:** medicine has a delivery deadline, and a drone that crashes or runs out of battery delivers nothing and may endanger people on the ground.

Search element	Definition for this problem
State	Current waypoint (plus, in an extended model, battery level and time of day)
Initial state	Depot / launch point S
Goal test	Drone is at the delivery point G (relief camp or hospital)
Actions	Fly to any neighbouring waypoint whose leg is currently not reported as blocked
Step cost c(n, n')	Energy / time to fly the leg = distance × terrain factor × weather factor (always > 0)
Heuristic h(n)	Straight-line (Euclidean) distance from n to G, scaled by the minimum cost per unit distance
Path cost g(n)	Sum of leg costs from S to n actually experienced or estimated
Objective	Minimise delay and risk, i.e. a weighted cost = time + λ × risk

The straight-line distance is **admissible** because no real route can be shorter than a straight line, provided it is multiplied by the cheapest possible cost per unit distance (best-case weather). It is also **consistent** (monotone), because by the triangle inequality h(n) ≤ c(n,n') + h(n'). These two properties matter for the A* discussion below.

Running example used throughout this answer

To make the comparison concrete, consider a small abstracted map. S is the depot, G is the hospital, and A, B, C, D are intermediate waypoints. Edge labels are the *current* flight costs (affected by weather) and the heuristic values are straight-line estimates to G.

Edge	S–A	S–B	A–C	A–D	B–D	C–G	D–G
Cost	2	5	10 (storm over this leg)	9	3	3	4

Node	S	A	B	C	D	G
h(n)	6	4	5	3	4	0

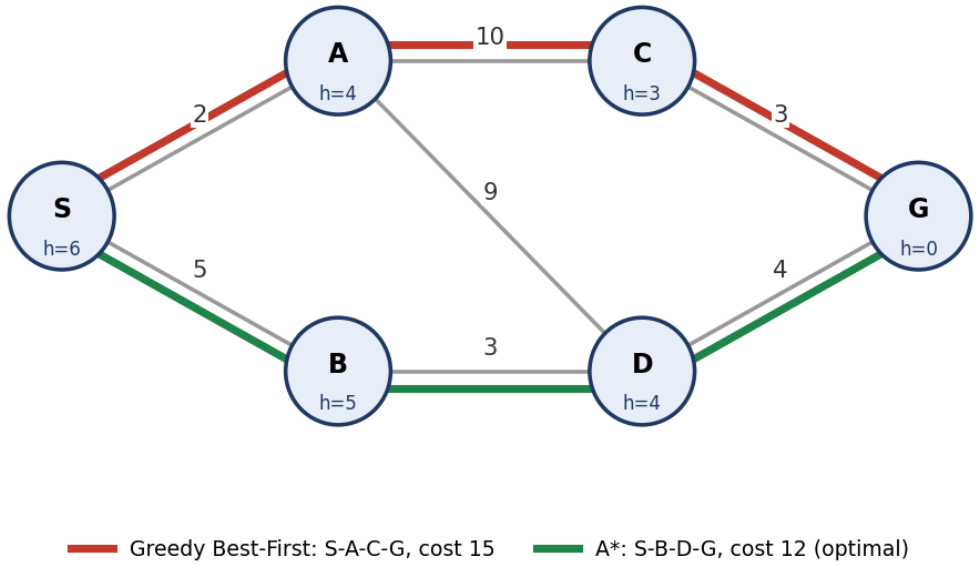


*Figure 1: Running example map (edge costs, heuristic h) with the routes chosen by Greedy Best-First Search and A**

Notice that the leg A–C is short in distance (so its heuristic looks attractive) but very expensive because of a storm cell. This is exactly the situation in which heuristic-only reasoning is dangerous.

1(i) Behaviour of Greedy Best-First Search (GBFS) and its strengths and weaknesses under uncertainty

How it behaves

Greedy Best-First Search keeps a priority queue (frontier) ordered only by the heuristic: **f(n) = h(n)**. At every step it expands the frontier node that *looks* closest to the goal in a straight line, and it ignores the cost already paid to get there. In the drone setting this means the drone always flies toward the waypoint that is geometrically nearest to the hospital, regardless of weather or terrain on that leg.

Trace on the running example. Frontier = {S}. Expand S: A (h=4) and B (h=5) are generated; A has the lower h, so A is expanded next. Expanding A generates C (h=3)

and D (h=4); C wins. Expanding C generates G (h=0), which is expanded next and the goal is found. The route is **S → A → C → G with cost 2 + 10 + 3 = 15**. Only four nodes were expanded, which is very fast, but the route flies through the storm cell and it is *not* the cheapest route (the best route costs 12, shown in 1(ii)).

Strengths under uncertainty

- **Very fast decisions.** It expands few nodes because it always dives toward the goal. With a deadline of seconds this keeps the planning delay low, and the first plan is available almost immediately.
- **Low memory and computation.** Only the heuristic is evaluated; no g-values have to be maintained or updated when weather changes the leg costs.
- **Cheap re-planning.** If a path is blocked, the drone simply removes the blocked node and picks the next best-looking node, so recovery is quick.
- **Robust to unknown costs.** When cost information is missing or noisy, a pure distance heuristic is still available, so GBFS never stalls for lack of data.

Weaknesses under uncertainty

- **Not optimal.** As the trace shows, it returned cost 15 when 12 was possible. In a flood, a 25% longer route can mean a missed delivery deadline or an exhausted battery.
- **Not complete in general graphs.** In graph search without an explored (closed) list it can loop between two nodes whose h-values look good. With an explored list it is complete for finite graphs, but it may still pick very long detours.
- **Ignores terrain and weather cost.** The heuristic is straight-line distance, so the algorithm cannot see that a short leg crosses a storm; it will confidently enter high-risk regions.
- **Trapped by dead ends.** A node that is close to the goal in a straight line but behind a flooded river looks excellent until the drone gets there. GBFS has no notion that the path so far was expensive.
- **Worst-case time and space are exponential,** $O(b^m)$, where m is the maximum depth of the search space.

1(ii) How A* handles the same situation: balancing cost and heuristic

A* orders the frontier by **f(n) = g(n) + h(n)**, where g(n) is the actual cost already incurred from the depot to n and h(n) is the estimated remaining cost. The first term makes the algorithm *cost-aware* (pessimistic about expensive past choices); the second makes it *goal-directed* (optimistic about promising futures). The balance is automatic: a node that has been expensive to reach is penalised even if it is geometrically close to the hospital, while a node with a cheap, safe past is rewarded.

Trace on the running example (consistent heuristic, so each node is expanded once):

Step	Expand	Generated nodes with g, h, f	Frontier after step (ordered by f)
1	S (f=6)	A: g=2,h=4,f=6; B: g=5,h=5,f=10	A(6), B(10)
2	A (f=6)	C: g=12,h=3,f=15; D: g=11,h=4,f=15	B(10), C(15), D(15)
3	B (f=10)	D improved: g=8,h=4,f=12 (via B, cheaper than 11)	D(12), C(15)
4	D (f=12)	G: g=12,h=0,f=12	G(12), C(15)
5	G (f=12)	Goal test succeeds	Stop

A* returns **S → B → D → G with cost 5 + 3 + 4 = 12**. It avoided the storm leg A–C because g(C) = 12 pushed f(C) up to 15. It expanded five nodes against GBFS's four, which is a small price for a 20% cheaper and safer route.

Properties that matter here

- **Optimal and complete** if h is admissible (tree search) or consistent (graph search). The straight-line heuristic satisfies both, so the returned route is the least-cost route *on the current map*.
- **Cost model integration.** Because g(n) uses real leg cost, changing weather simply changes edge weights; A* needs no change in logic.
- **Risk-aware cost function.** Step cost can be defined as time + λ × risk, with risk raised in turbulent or flooded cells. A* then trades a slightly longer but safer corridor against a shorter but dangerous one automatically.
- **Weakness:** in the worst case it expands exponentially many nodes and keeps them all in memory, $O(b^d)$ space. For a large region the planning delay can be noticeable; mitigations are weighted A* ($f = g + w \cdot h$, $w > 1$), a hierarchical (coarse-to-fine) map, or an anytime variant.

1(iii) Impact of heuristic accuracy on both algorithms

Let h*(n) denote the true optimal remaining cost. The ratio h(n)/h*(n) describes accuracy.

Heuristic quality	Effect on Greedy Best-First	Effect on A*
Perfect (h = h*)	Goes straight to the goal along an optimal path; ideal.	Expands only nodes on the optimal path; ideal and optimal.
Accurate but slightly under-estimating (admissible)	Fast and usually good, but still no optimality guarantee because g is ignored.	Optimal; expands few nodes. The closer h is to h*, the fewer nodes expanded.
Very weak (h ≈ 0)	Degenerates to an uninformed, random-looking search; may wander.	Degenerates to Dijkstra / uniform-cost search; still optimal but slow.

Heuristic quality	Effect on Greedy Best-First	Effect on A*
Over-estimating (inadmissible), e.g. a terrain penalty that is too pessimistic	Barely affected; it only needs a ranking. A misleading ranking still sends the drone to wrong places.	Loses the optimality guarantee; may skip the best route, but often runs faster (the principle behind weighted A*).
Misleading (looks good in a bad region, like the storm leg)	Severe damage: the algorithm commits to the wrong region and has no cost signal to recover.	Self-correcting: g(n) grows as the drone pays the high cost, so f rises and search switches to other branches.

Key insight. Greedy search depends completely on the *ranking* produced by the heuristic, so its quality is a direct function of heuristic quality. A* is more tolerant: as long as h never over-estimates, the g-term corrects the errors. In this scenario the straight-line heuristic ignores terrain and weather, so it systematically *under-estimates* the true remaining cost in bad weather. Under-estimation is safe for A* (admissible) but it means A* explores more nodes than it would with a weather-aware heuristic. An improved heuristic h'(n) = straight-line distance × (minimum local weather factor) remains admissible and is tighter, so A* becomes faster with no loss of optimality.

1(iv) How dynamic changes (blocked paths) affect search performance

Flooding changes the graph during execution, so a plan that was optimal at take-off may become invalid. The effects differ by algorithm.

Aspect	Greedy Best-First	A*
Detecting invalid plan	Plan is checked leg by leg; blocked next node is dropped.	Same, but g-values of descendants and the optimal path become stale.
Re-planning cost	Very cheap: choose next lowest h from the frontier.	Higher: edge costs changed, so g and f must be recomputed; basic A* restarts from the drone's current position.
Quality after re-plan	May be pushed into a long detour with no cost awareness.	Still optimal for the updated map, since it re-optimises with new costs.
Risk of oscillation	High: a node can be dropped and re-added as updates arrive.	Lower: costs encode the penalty of changes.
Best incremental variants	Not needed; already local.	D* Lite, LPA* and Anytime Repairing A* reuse earlier search effort and update only affected nodes.

Example. Suppose the leg D–G becomes blocked while the drone is at B on the route S → B → D → G. A* re-plans from B on the updated map: the only remaining route to G is B → D → A → C → G with cost 3 + 9 + 10 + 3 = 25, so the drone knows the true price of the blockage and can decide whether battery allows it or whether to divert to an alternative landing point. Greedy search, standing at D, would simply move to the neighbour with the lowest h (C is not adjacent, so A) without comparing total cost. In both cases performance degrades as the **rate of change** increases: if blockages

occur faster than re-planning finishes, the plan is always stale. This is why a pure offline plan is insufficient and the execution loop must be **plan → fly one leg → sense → update map → re-plan if needed**.

Other practical consequences of dynamic change are: (a) **stale heuristic information**, because the straight-line distance does not know about the new obstruction; (b) **possible loss of completeness**, since the goal could become temporarily unreachable and the algorithm must report failure and request an alternative landing point; (c) **battery constraint**, because every re-route spends energy that must be reserved as a safety margin; and (d) **information delay**, since partial updates mean the drone may fly into an unreported blockage and have to turn back.

1(v) Recommendation and justification

Recommendation
Use **A* (with a weather- and risk-aware cost function and an admissible, tightened heuristic), executed in a receding-horizon loop with incremental re-planning (D* Lite / LPA* style), and a greedy fallback only if the planning deadline is about to be missed.**

Criterion	Greedy Best-First	A*	Verdict
Real-time constraint	Fastest planning	Slightly slower; acceptable with weighted/incremental variants and a coarse map	Slight edge to Greedy, but A* is fast enough
Optimality (minimum delay)	Not guaranteed; 15 vs 12 in the example	Optimal with admissible h	A*
Safety (risk of entering storms or dead ends)	Ignores cost and risk	Risk can be built into g(n) and the cost function	A* strongly
Handling blocked paths	Cheap but blind	Incremental versions re-use work and still stay optimal	A* (D* Lite)
Handling uncertain information	Depends entirely on heuristic ranking	Self-correcting through g(n)	A*
Resource use	Low	Higher memory	Greedy; mitigated by weights/hierarchy

Justification

1. **Safety dominates.** A wrong choice may lose the drone and the medicine. Only a cost-aware algorithm can represent weather and risk, and A* does so through g(n).
2. **Optimality is worth the small time premium.** The extra expansion of one node in the example (five versus four) bought a 20% saving; at scale the same ratio of cost saving is typically larger and delay is mission-critical.
3. **Real-time requirements are met by engineering,** not by abandoning optimality: (a) weighted A* with w between 1.2 and 1.5 gives bounded sub-optimality (cost ≤ w × optimal) with large speed-ups; (b) a hierarchical map

reduces the number of nodes; (c) an *anytime* approach returns a valid route immediately and improves it while the drone flies; (d) D* Lite repairs the plan after each new blockage by re-using previous work.

4. **Fail-safe layer.** If a re-plan cannot be completed within the control-cycle deadline, the drone falls back to the greedy choice *restricted to nodes that satisfy safety rules* (battery reserve, no-fly zones) or to a pre-computed return-to-base path.

Conclusion. Greedy Best-First Search is attractive for speed but is unsafe and sub-optimal in a flood scenario. A* with incremental re-planning provides the best compromise between delay, optimality and safety, and is therefore the recommended algorithm.

Question 2: Smart-City Traffic Signal Optimisation

2.0 Understanding the scenario

A smart city wants an AI controller that sets the timing of traffic signals at hundreds of intersections. The controller must minimise **total waiting time**, **fuel consumption** and **congestion**. The difficulty has four sources: the number of possible timing combinations is astronomically large; the objectives interact (a green wave on one road can create a queue on a crossing road); traffic changes from minute to minute, so the optimal setting is a moving target; and simple improvement methods can get stuck at a solution that is good locally but poor city-wide.

2(i) Formulating the problem as an optimisation problem; why traditional search is inefficient

Formal formulation

Component	Definition
Decision variables	For each intersection i (i = 1…N): cycle length C_i, green split $g_{i,k}$ for each phase k, and offset θ_i relative to a reference signal (for coordination with neighbours).
State / solution x	The vector x = (C_1, $g_{1,1}$, …, θ_1, …, C_N, …) of all timing parameters.
Domain	Discrete values: cycle 40–120 s in 5 s steps, green splits in 2 s steps, offsets 0 to C in 5 s steps; subject to minimum green (pedestrian safety) and maximum cycle limits.
Objective function	Minimise $F(x) = w_1 \cdot W(x) + w_2 \cdot E(x) + w_3 \cdot Q(x)$, where W = total vehicle waiting time, E = fuel consumed (idling and stop-start), Q = congestion (e.g. sum of squared queue lengths or number of saturated links).
Evaluation	F(x) is computed with a traffic simulator or a flow model (for example a cell-transmission model) using live detector data.
Constraints	Minimum / maximum green, pedestrian crossing times, emergency vehicle priority, fixed cycle on coupled intersections.
Neighbourhood	Change one parameter by one step at one intersection (or at two adjacent ones).

This is a **combinatorial, multi-objective, non-linear and dynamic optimisation problem**. The multi-objective part is reduced to a single number with weights w_1, w_2, w_3 chosen by city policy (for example 0.5, 0.2, 0.3) or it can be handled as a Pareto problem.

Why the search space is huge

Assume 300 intersections, each with 4 parameters that take about 40 discrete values. The number of complete settings is $40^{1200} \approx 10^{1900}$. For comparison, the number of atoms in the observable universe is about 10^{80}. Even if each evaluation took a microsecond, enumerating the space is impossible.

Why traditional (uninformed / classical informed) search is inefficient

- **Breadth-first, depth-first and uniform-cost search** enumerate states systematically; their time and memory are exponential in the number of variables ($O(b^d)$). They cannot finish in the seconds available.
- **A* needs an admissible heuristic and a goal state.** Here there is no single goal state: we want the *best* value of a function, not a path to a known goal. No cheap, accurate heuristic for "remaining improvement" exists.
- **The solution is the state, not the path.** The path used to reach a configuration is irrelevant, so storing paths and frontiers wastes memory. Local search methods that keep only the current state are much more suitable.
- **Dynamic environment.** Traffic changes, so any full search result is obsolete by the time it finishes; the controller needs an *anytime* method that gives a good answer immediately and keeps improving it.
- **Strong interactions between variables** (neighbouring intersections) make it impossible to optimise each junction independently and then combine results.

Conclusion of 2(i)
Because the state space is vast, there is no explicit goal, and solutions must be refreshed continuously, **local search / meta-heuristic optimisation** (hill climbing, simulated annealing, genetic algorithms) is the appropriate family, not systematic search.

2(ii) How Hill Climbing works in this scenario and its limitations

Algorithm

Hill climbing (steepest-ascent, here steepest-*descent* because we minimise cost) starts from a current timing plan x, evaluates all neighbours, and moves to the neighbour with the lowest F. It stops when no neighbour improves F.

```
function HILL-CLIMBING(initial_plan x):
    current <- x
    loop:
        neighbours <- all plans obtained by changing ONE green split / cycle / offset by one step
        best <- neighbour in neighbours with minimum F(.)
        if F(best) >= F(current): return current      # local optimum
        current <- best
```

Real-world operation. The controller begins with the plan that is currently running (for example the fixed-time plan). It tries lengthening the green of the north-south phase at junction 17 by 2 s, shortening it, changing the offset by 5 s, and so on for every junction. It adopts the change that reduces the combined cost most and repeats. Each iteration is cheap because only the parts of the network around the changed junction need to be re-evaluated, so it gives rapid improvement.

Strengths

- Very simple, low memory (only the current plan is stored), fast, and an **anytime** method.

- Well suited to **incremental, online fine tuning**: every small improvement can be applied directly to the real network.
- Works naturally on a discrete neighbourhood.

Limitations

- **Gets stuck at local optima**: it halts at the first plan where no single small change helps, even though a far better plan exists.
- **Cannot cross plateaus**: if neighbours have equal cost it has no direction (it may wander or stop).
- **Dependent on the starting point**: different starting plans lead to different final quality.
- **Single-parameter moves are too myopic** for variables that must change together (ridges).
- **No guarantee of global optimality and no convergence estimate**; and it cannot adapt on its own if the landscape shifts, since it has stopped.

2(iii) Local maxima, plateaus and ridges with real-world interpretation

Because hill climbing is described in terms of an *objective landscape*, we translate the three classic difficulties into traffic terms (here "maximum" means the best value of a quality score, which is equivalent to a minimum of the cost F).

Landscape feature	Technical meaning	Real-world traffic interpretation	Consequence
Local optimum (maximum of quality)	A state better than all neighbours but worse than the global best.	The timing plan favours the main arterial at every junction. Any single change (more green for a side road) increases waiting on the arterial more than it saves, so the search stops. A better city-wide plan would re-time a whole corridor and use a different cycle length.	Search ends with avoidable congestion remaining.
Plateau / flat region	A region where neighbours have equal F, giving no gradient.	Off-peak night traffic: changing a green split by 2 s makes no measurable change to waiting time because few vehicles are present. Also, saturated junctions where queues are so long that small timing changes do not clear them.	Algorithm cannot choose a direction; it wanders or terminates early.
Ridge	A narrow high-quality path along which improvement needs two or more variables to change simultaneously.	A green wave: to improve flow on a corridor the offset of junction 5 and junction 6 and the cycle length must change *together*. Changing any one of them alone worsens the flow, so each single-move neighbour looks worse than the current plan.	Algorithm cannot climb the ridge because all single steps point downhill.

Landscape feature	Technical meaning	Real-world traffic interpretation	Consequence
Shoulder	A plateau that eventually rises.	Slow growth in demand: small timing change has no effect until the volume crosses a threshold and then has a sudden effect.	Algorithm stops before the rise.
Moving landscape	The objective changes with time.	Rush-hour build-up, an accident, a football match: a plan at the local optimum becomes bad within minutes.	Static local optimum becomes obsolete.

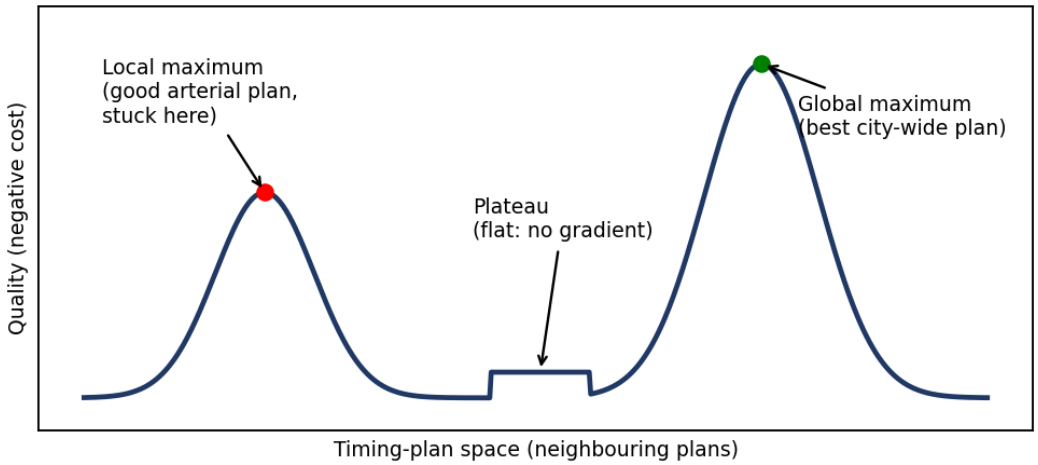


Figure 2: Objective landscape showing a local maximum, a plateau and the global maximum

Common remedies within the hill-climbing family are **random restarts** (start from many different plans and keep the best), **stochastic hill climbing** (pick a random improving move), **first-choice hill climbing**, **sideways moves** (accept equal-cost moves for a limited number of steps to cross plateaus) and **compound moves** (change a pair of variables together, such as offsets of adjacent junctions, to climb ridges). These improve it but do not give a systematic way of escaping deep local optima, so stronger meta-heuristics are explained next.

2(iv) How Simulated Annealing or Genetic Algorithms overcome these limitations

Simulated Annealing (SA)

SA is inspired by cooling of metals. It allows **uphill (worse) moves with a probability that decreases over time**, so that early in the search it can escape local optima, and later it settles like hill climbing.

```
function SIMULATED-ANNEALING(x0, T0, alpha, T_min):
    x <- x0 ; T <- T0
    while T > T_min:
        x'    <- random neighbour of x
        delta <- F(x') - F(x)                 # F is cost (minimise)
        if delta < 0:            x <- x'      # always accept improvements
        else if random() < exp(-delta / T):   x <- x'   # accept worse with probability
        T <- alpha * T                        # cooling, e.g. alpha = 0.95
    return best x seen
```

- **Local optima:** a worse plan is accepted with probability $e^{-\Delta/T}$. At high T (say T = 100) a move that adds 10 units of cost is accepted with probability $e^{-0.1} \approx 90\%$; at low T (T = 2) the same move has probability $e^{-5} \approx 0.7\%$. The controller therefore leaves a deceptive valley early but converges later.
- **Plateaus:** random neighbour selection with equal cost gives a random walk across the flat region instead of stopping.
- **Ridges:** a temporary worsening (changing one offset alone) can be accepted, after which the second variable improves, so the ridge is crossed.
- **Properties:** with a sufficiently slow logarithmic cooling schedule, SA converges to the global optimum with probability 1 (in practice faster schedules are used). It is simple, low-memory and anytime; it needs tuning of T_0 and α.
- **Traffic usage:** re-heating (raising T slightly) when detectors report a change of pattern lets the search escape the plan that has become obsolete.

Genetic Algorithm (GA)

A GA maintains a **population** of candidate timing plans (chromosomes) and evolves it through selection, crossover and mutation.

GA element	Traffic-signal design
Chromosome	Concatenated timing parameters, e.g. [C_1, $g_{1,1}$, $g_{1,2}$, θ_1 | C_2, ...]. A natural variant is a block per city district.
Fitness	1 / (1 + F(x)) from a simulator or a flow model (higher is better).
Selection	Tournament or roulette-wheel selection favouring plans with lower waiting time, fuel and congestion.
Crossover	Swap the timing of whole districts or corridors between two parents so that coordinated sub-plans (green waves) remain intact: this directly deals with ridges.
Mutation	Random small change of a green split or an offset (probability about 1–2% per gene) to maintain diversity and explore.
Elitism	Keep the best few plans unchanged in the next generation so that quality never decreases.
Termination	Fixed number of generations, time budget or convergence of the best fitness.

- **Local optima:** the population explores many regions simultaneously; different individuals sit in different valleys, so the whole population is unlikely to be trapped together.
- **Plateaus:** mutation and crossover produce jumps of different size so the search is not restricted to a one-step neighbourhood.
- **Ridges:** crossover combines good building blocks (sub-corridors with good coordination) discovered independently, so variables that must change together are recombined as a unit.
- **Parallelism:** fitness evaluations are independent and can run on many processors or on edge servers per district.

- **Weaknesses:** many fitness evaluations (computationally heavier than SA), parameter tuning (population size, mutation rate), and slower response when traffic changes abruptly unless the population is kept warm.

Property	Hill Climbing	Simulated Annealing	Genetic Algorithm
Escapes local optima	No	Yes (probabilistic uphill moves)	Yes (population diversity)
Crosses plateaus	Poorly	Yes (random walk)	Yes (mutation/crossover)
Handles ridges	No	Partly	Well (crossover of blocks)
Memory	Very low	Very low	Higher (population)
Evaluations per step	Many (all neighbours)	One	Population size
Speed of reaction to changes	Very fast	Fast	Slower, but improved with warm start
Parallelisable	Limited	Limited (multi-chain)	Highly
Global quality	Low	High	High

2(v) Recommended approach and justification

Recommendation
A **hierarchical hybrid**: a **Genetic Algorithm** (optionally seeded and refined by Simulated Annealing) to compute good city-wide base plans every few minutes or offline for typical traffic patterns, combined with **fast, local hill-climbing / SA fine-tuning per district in real time** (a memetic algorithm). The weights of the three objectives are set by city policy.

Justification based on scalability

- **Decomposition.** The city is split into districts / corridors with limited interaction. A GA or SA runs independently on each district (and in parallel on edge servers), and a coordination layer only aligns the border junctions. This keeps the problem size per optimiser small and scales almost linearly with the number of junctions.
- **Evaluation cost.** Meta-heuristics need no gradient and no explicit goal; they use the simulator as a black box, so the model can be as detailed as the city can afford.
- **Anytime behaviour.** At every moment the best plan found so far is deployable, so the system never has to wait for a full search.

Justification based on adaptability

- **Warm starts.** When traffic changes, the previous best population (GA) or plan (SA) is used as the starting point, so the system adapts in seconds rather than starting from scratch.
- **Re-heating.** A change detector (sudden rise in queue length) increases the SA temperature and the GA mutation rate to re-open exploration; when the situation is stable, it cools down to exploit.

- **Diversity.** The GA population retains plans suited to different traffic regimes (morning peak, evening peak, event), so it can switch quickly.
- **Safety limits.** Minimum green times, pedestrian clearance and emergency-vehicle priority are hard constraints in the search (infeasible plans are repaired or rejected), and a new plan is introduced gradually (limited step per cycle) to avoid confusing drivers.

Conclusion. Pure hill climbing is cheap but gets trapped; pure GA is powerful but slower to react; pure SA is simple but handles only one solution. A hybrid that uses GA/SA for global exploration and hill climbing for local refinement gives the best balance of solution quality, scalability to hundreds of intersections and adaptability to changing traffic.

Question 3: Autonomous Mars Rover in Unknown Terrain

3.0 Understanding the scenario

A rover on Mars must reach sample sites, avoid hazards (craters, loose rocks, steep slopes) and operate with limited sensors and no complete map. Communication with Earth has a delay of roughly 4 to 24 minutes one way, so the rover cannot be remotely driven step by step and must decide for itself. Its knowledge grows as it moves, so planning and acting are interleaved.

Environment property	Mars rover
Observable?	Partially: cameras and lidar see only a few metres to tens of metres; obstacles behind rocks are hidden.
Deterministic?	No: wheel slip on sand, sensor noise and localisation drift make outcomes uncertain.
Static?	Mostly static terrain but dynamic knowledge, plus dust storms, changing light and shadows.
Known?	No: the map is initially unknown (an exploration problem).
Episodic / sequential?	Sequential: a wrong move (stuck in sand) affects all later actions and may be irreversible.
Safety	Mission-critical: a mistake cannot be repaired by a human nearby.

3(i) Why the problem requires an online search agent instead of offline search

An **offline search agent** computes a complete plan (a path from start to goal) *before* acting, using a model that is fully known. An **online search agent** interleaves computation and action: it takes an action, observes the percepts, updates what it knows, and then decides on the next action. It is the right choice when the environment is unknown or the model is incomplete.

- **No complete map.** Offline search requires the successors and costs of every state in advance. The rover does not know which cells are craters until it sees them, so there is nothing to search over at the start.
- **Exploration is part of the task.** The agent only discovers the actions' outcomes by trying them (what is beyond that ridge?). This is the classic definition of an online search problem.
- **Percepts change the plan.** Each new image can invalidate the earlier plan; a pre-computed route would be wrong as soon as an unseen boulder field appears.
- **Communication delay.** Earth-based planning is too slow for real-time hazard avoidance; autonomy must be on board.
- **Irreversibility.** Some actions (driving into soft sand) cannot be undone. An online agent can test the surroundings cautiously and avoid such states, whereas an

offline plan based on guesses cannot.

- **Limited computation and energy.** Planning the entire map in advance with onboard hardware is infeasible; planning only a local horizon is affordable.

Online agents are usually evaluated by the **competitive ratio**: the cost of the path actually travelled divided by the cost of the shortest path that would have been followed if the whole environment were known. A smaller ratio means a more efficient explorer. In some environments (dead ends that cannot be left) no online algorithm can guarantee a bounded ratio, which is why safety must be handled explicitly.

3(ii) Challenges of partial observability and dynamic environments

Partial observability

- **Hidden hazards.** A crater rim or a drop behind a rock cannot be seen from the current position. The rover must reason about what may be there (unknown space) and treat unexplored cells conservatively.
- **Sensor noise and limited range.** Stereo cameras are inaccurate at distance, shadows hide depth, and dust reduces visibility. Percepts therefore carry uncertainty, so the agent maintains probabilities (e.g. P(cell is obstacle) = 0.7).
- **Perceptual aliasing.** Two different places can look alike (similar rocks), which makes recognising a previously visited location hard and can cause loops.
- **Localisation uncertainty.** Wheel odometry drifts on sand; without GPS the rover needs visual odometry and landmarks. Errors accumulate, so the map becomes inconsistent.
- **Belief-state explosion.** A true belief-state search over all possible maps is intractable, so approximations (occupancy grids, particle filters) are necessary.
- **Exploration versus exploitation.** The rover must balance going to the goal (collecting samples) with gathering information (looking around), because information has a cost in time and energy.

Dynamic environment

- **Changing conditions.** Dust storms reduce solar power and visibility; lighting changes make hazard detection different at different times of the day; wind can move loose sand and change the terrain.
- **Stale knowledge.** A cell seen as safe two hours ago may no longer be safe, so the map needs time stamps and confidence decay.
- **Continual re-planning.** The plan must be repaired whenever a new obstacle is detected, within the real-time control cycle.
- **Resource dynamics.** Battery, temperature and time of day impose changing constraints (night-time survival heating, communication windows).

- **Safety.** There is no recovery from some failures, so uncertainty must lead to caution (stop and sense again, back up, choose a safer detour).

3(iii) How the rover builds and updates its internal model

The rover keeps an **internal model** consisting of a map, its own pose and estimates of costs. This model is the only way an online agent can plan in an unknown world. It is built in a perception–update–plan–act cycle:

1. **Sense.** Cameras, lidar/stereo depth, inertial measurement unit and wheel encoders produce percepts (distance to obstacles, slope, surface type, orientation).
2. **Localise.** Visual odometry and landmark matching estimate the rover's pose (x, y, heading). A *SLAM* approach (Simultaneous Localisation and Mapping), using for example a particle or Kalman filter, corrects drift while the map is being built.
3. **Map update.** Percepts are inserted into an **occupancy grid** (or a cost map). Each cell stores a probability of being occupied and a traversability cost. Update rule (Bayesian log-odds): $l_t(c) = l_{t-1}(c) + \text{inverse_sensor_model}(c) - l_0$. Cells never seen are labelled *unknown* and receive a conservative cost.
4. **Result table.** The agent stores *result[s, a] = s'* (the outcome observed after action a in state s) and, for learning real-time agents, an estimated cost-to-goal *H[s]* for each visited state.
5. **Plan.** A planner (A*, D* Lite or LRTA*) computes the best path through the *currently known* map plus optimistic or cautious assumptions about unknown cells.
6. **Act and verify.** The rover executes only the first few metres, then senses again; if the percepts contradict the model, the model is corrected (replace belief with new evidence) and the plan is repaired.

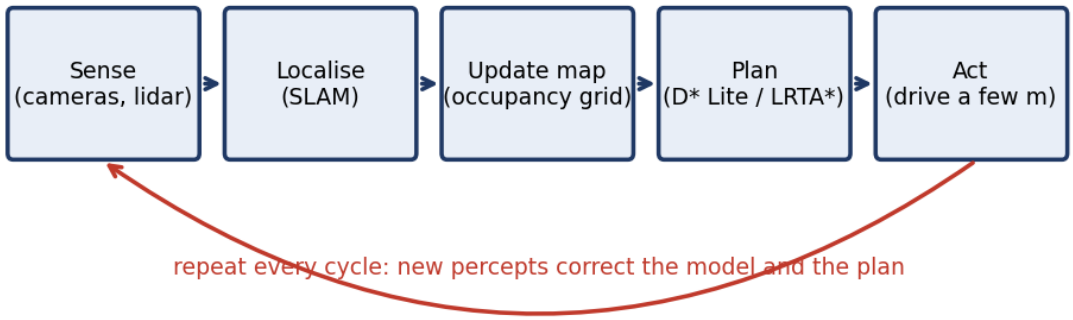


Figure 3: Online agent cycle of the rover (sense – localise – update map – plan – act)

Illustration with a grid

```
Known map after 3 steps (R = rover, # = obstacle seen, . = free, ? = unknown, G = goal)

   ? ? ? ? ? ?        Step 1: sense -> cells ahead free
   ? . . # ? ?        Step 2: obstacle '#' detected -> mark occupied, raise cost near it
   ? . R . ? ?        Step 3: replan around '#' using D* Lite, only affected nodes updated
   ? . . . ? G        Unknown cells (?) get cautious cost; known free cells get low cost
```

Rover grid: dark = detected obstacle, dashed = re-planned path

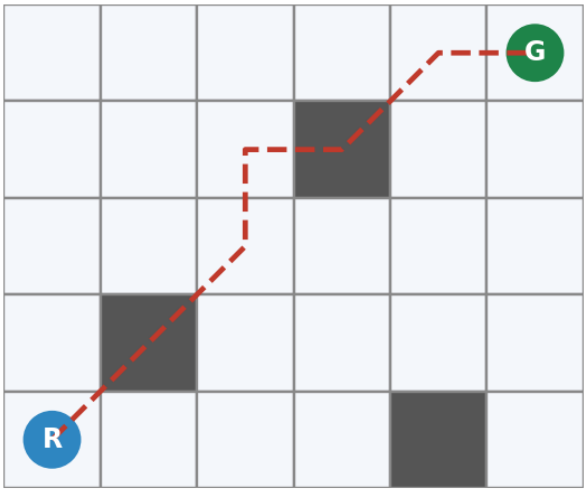


Figure 4: Rover grid map; the dashed path is re-planned around newly detected obstacles

Learning real-time A* (LRTA*) is a simple online algorithm that shows how the model is updated while acting. It stores a heuristic table H(s). At each step: (1) look at all neighbours s' of the current state s, (2) choose the action minimising c(s,a,s') + H(s'), (3) update H(s) ← min over actions of [c(s,a,s') + H(s')], (4) move. The repeated update makes a misleading heuristic self-correct: the cost estimate of a local trap (a pocket surrounded by rocks) rises each time the rover visits it, until the rover leaves and does not return.

```
function LRTA*-AGENT(s'):                    # s' = percept identifying current state
    if GOAL-TEST(s'): return stop
    if s' is new:  H[s'] <- h(s')            # initial heuristic, e.g. straight-line distance
    if s (previous state) is not null:
        result[s, a] <- s'
        H[s] <- min over actions b in ACTIONS(s) of  COST(s, b, result[s, b]) + H[result[s, b]]
    a <- action b in ACTIONS(s') minimising  COST(s', b, result[s', b]) + H[result[s', b]]
          (unexplored actions are assumed to cost h(s'), i.e. optimistic -> encourages exploration)
    s <- s' ; return a
```

For larger maps the **D* Lite** algorithm is more efficient: it searches backward from the goal and, whenever the rover discovers a new obstacle, repairs only the nodes whose cost values have changed, rather than re-planning from scratch.

3(iv) Comparison of online search with uninformed and informed search strategies

Feature	Uninformed (BFS, DFS, UCS) – offline	Informed (Greedy, A*) – offline	Online search (LRTA*, D* Lite, online DFS)
Knowledge required	Complete model of states, actions, costs	Complete model plus a heuristic	Only current percepts; the model is learned
When computation happens	Entire plan before acting	Entire plan before acting	Interleaved with acting (plan–act–sense)
Handles unknown terrain	No	No	Yes
Handles changing information	Needs full restart	Needs full restart	Repairs/updates the plan incrementally
Optimality	BFS (equal costs) and UCS optimal	A* optimal with admissible h	Not guaranteed; competitive ratio may be unbounded in the worst case
Completeness	BFS/UCS complete for finite graph	A* complete	Complete in finite, safely explorable spaces
Expansion order	Any node in frontier (can jump around the map)	Best f-value anywhere in frontier	Only **local** expansion; the agent must physically be at the node
Memory	Large frontier	Large frontier	Compact map and H-table of visited states
Typical problems	Puzzles, known maps	Route finding with a known map	Robot exploration, game NPCs, unknown mazes
Safety under uncertainty	Not applicable	Not applicable	Can sense before acting, avoid irreversible actions

Key distinction. Offline algorithms may expand a node far away from the current position, because "expansion" is merely a computation. An online agent can expand only the node where it physically is, so it needs an order of visiting that is local. For example, depth-first search can be made online by *physically* backtracking: the rover keeps a table of unexplored actions in each visited state and, when a state is exhausted, returns to the state from which it came. This requires that actions are reversible; otherwise the rover cannot backtrack, which is a safety issue on Mars.

Cost implication. Offline A* on a known map produces the shortest route. The rover however usually has to travel extra distance to discover obstacles; the extra ratio (travelled cost ÷ optimal cost) is the price of ignorance. Informed heuristics (distance to the goal or to the nearest unexplored frontier) reduce this price relative to uninformed online exploration.

3(v) Most suitable approach considering uncertainty, adaptability and safety

Recommendation
An **online, informed, incremental replanning agent**: a **D* Lite (or LRTA*-style) planner** working on a probabilistic occupancy / cost map built by SLAM, with a **cautious, safety-first layer** (hazard buffers, risk-weighted costs, stop-and-resense rules, return-to-safe-state behaviour) and a **frontier-based exploration strategy** that balances information gain with progress toward sample sites.

Justification

- **Uncertainty.** The map is probabilistic: each cell has an occupancy probability. The planner uses risk-weighted costs, e.g. cost(c) = base + K·P(obstacle) so that uncertain areas are avoided unless necessary. Unknown cells are given a conservative cost near hazards and an optimistic cost far from them to promote exploration without recklessness.
- **Adaptability.** D* Lite re-uses previous search results and repairs only the affected parts of the plan, making it fast enough for onboard computers with limited processing power. LRTA* style learning of H values lets the rover escape local traps by itself.
- **Safety.** (a) Inflate each obstacle by a safety margin equal to the rover's radius plus the localisation error; (b) limit slope and roughness by terrain-classification rules; (c) drive short segments then re-sense (look before you leap); (d) keep a *safe return path* so the rover never enters a region it cannot leave (enforces the safely-explorable assumption); (e) hand over to a safe mode (stop and request ground control) when uncertainty exceeds a threshold or battery is low.
- **Efficiency.** A goal-directed heuristic (straight-line distance to the sample site) keeps the exploration focused; a frontier-based component picks the next most informative unexplored boundary when the direct route is blocked.
- **Why not the alternatives.** Offline uninformed or informed search needs a complete map and cannot start; pure greedy online movement can be trapped in a crater pocket; an unbounded random exploration wastes limited energy.

Conclusion. Because the rover operates in an unknown, partially observable and slowly changing world where mistakes are irreversible, an online informed search agent with incremental re-planning and explicit safety rules provides the best trade-off between adaptability, efficiency and safety.

Question 4: Automatic University Exam Timetabling

4.0 Understanding the scenario

The university must assign every exam to a time slot and an exam hall so that a set of hard rules is respected: no student has two exams at the same time, hall capacity is not exceeded, prerequisite ordering is respected, invigilating faculty are available, and students with special accommodations (extra time, a separate room, a ground-floor hall) are served. Because the choices of different exams interact through shared students, shared halls and shared faculty, the problem is a classic **Constraint Satisfaction Problem (CSP)**. Its size grows quickly: with E exams, S slots and H halls there are $(S \times H)^E$ possible assignments.

4(i) Formulation as a CSP: variables, domains and constraints

A CSP is a triple **(X, D, C)**: a set of variables X, a domain D_i of values for each variable, and a set of constraints C that specify allowed combinations of values.

Variables

For each exam e_i (i = 1…E) there are two decision variables: **Slot(e_i)**, the time slot in which the exam is held, and **Hall(e_i)**, the room(s) in which it is held. (They can be combined into a single variable whose value is a pair (slot, hall); large exams that need several halls are split into sections $e_{i,1}$, $e_{i,2}$, … each being a variable.) A further variable **Invigilator(e_i)** can be included if staff allocation is part of the problem.

Domains

Variable	Domain	Explanation
Slot(e_i)	{1, 2, …, S}, e.g. 10 days × 2 sessions = 20 slots	Discrete finite time slots of the exam period.
Hall(e_i)	{H_1, …, H_m} filtered by capacity	Initially all halls; unary constraints remove halls that are too small or lack required facilities.
Invigilator(e_i)	Set of faculty qualified/available for the exam	Reduced by faculty availability constraints.

Constraints

No.	Type	Constraint (formal)	Explanation
C1	Binary, hard	If exams e_i and e_j share at least one student, then Slot(e_i) ≠ Slot(e_j)	No student has overlapping exams. Defines a *conflict graph* (exams are nodes, shared students are edges), so the problem includes graph colouring.
C2	Binary, hard	If Slot(e_i) = Slot(e_j) then Hall(e_i) ≠ Hall(e_j) (all-different on halls per slot)	Two exams cannot use the same hall at the same time.
C3	Unary, hard	Capacity(Hall(e_i)) ≥ Students(e_i), or the sum of capacities of assigned halls ≥ Students(e_i)	Limited exam halls and their sizes.

No.	Type	Constraint (formal)	Explanation
C4	Binary, hard	If subject A must precede subject B, Slot(A) < Slot(B)	Precedence constraints (e.g. foundation before advanced paper).
C5	Unary / binary, hard	$Slot(e_i) \in Available(Invigilator(e_i))$; an invigilator cannot supervise two exams in the same slot	Faculty availability.
C6	Unary, hard	For students with accommodations: exam is in a ground-floor or quiet hall, and slot duration ≥ extended time; no two consecutive exams for the same student	Special accommodation.
C7	Global	Number of exams in a slot ≤ number of halls (the number of simultaneous exams in a slot is bounded)	Global capacity constraint (AllDiff-type).
C8	Soft	Minimise the number of students with two exams on the same day or in consecutive slots; spread exams of large courses	Quality preferences; handled as an objective (weighted penalty) after hard constraints are satisfied.

Constraint graph. Nodes are the exam variables and an arc joins two variables that appear together in a binary constraint. The graph can be partitioned into loosely connected components (for example faculties that share no students), which can be solved independently. This property is used in the scalability strategy below.

Small worked instance

Take five exams A, B, C, D, E; three slots {1, 2, 3}; two halls. Shared-student conflicts: A–B, A–C, B–C, C–D, D–E. Precedence: A before D. The conflict graph contains a triangle (A, B, C), so three distinct slots are needed for them. A valid solution is A = 1, B = 2, C = 3 (all different), D = 1 is forbidden by C4 (A < D requires D > 1) so D = 2 (D conflicts with C only, and 2 ≠ 3, fine), E = 1 or 3 (E conflicts only with D = 2).

Exam	A	B	C	D	E
Slot	1	2	3	2	1
Hall	H1	H1	H1	H2	H1

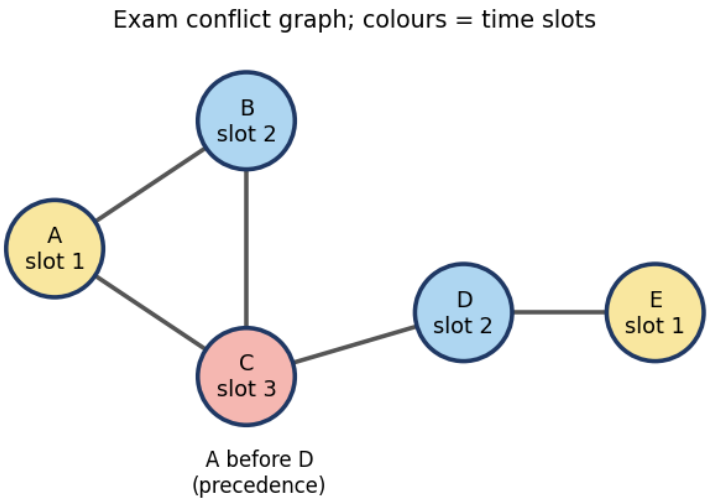


Figure 5: Conflict graph of the worked instance with a valid slot assignment

Check: A–B (1≠2), A–C (1≠3), B–C (2≠3), C–D (3≠2), D–E (2≠1) all satisfied; A < D (1 < 2); in slot 2 exams B and D use different halls (H1, H2). The assignment is complete and consistent.

4(ii) How backtracking search works in this scenario

Backtracking search is a depth-first search that assigns **one variable at a time**, checks that the partial assignment does not violate any constraint, and *backtracks* (undoes the latest assignment and tries the next value) when it reaches a dead end. Because variable assignments are commutative (the order of assigning exams does not change the set of solutions), only one variable is chosen at each level of the tree, so the tree has depth E and branching factor |D|, not E!·$|D|^E$.

```
function BACKTRACKING-SEARCH(csp): return BACKTRACK({}, csp)

function BACKTRACK(assignment, csp):
    if assignment is complete: return assignment
    var <- SELECT-UNASSIGNED-VARIABLE(csp, assignment)        # e.g. MRV + degree
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp):   # e.g. LCV
        if value is consistent with assignment:
            add {var = value} to assignment
            inferences <- INFERENCE(csp, var, value)           # forward checking / AC-3
            if inferences != failure:
                add inferences to assignment
                result <- BACKTRACK(assignment, csp)
                if result != failure: return result
            remove {var = value} and inferences from assignment
    return failure
```

Trace on the worked instance. Order: A, B, C, D, E; values tried in ascending order.

Step	Assignment	Check	Result
1	A = 1	No constraints yet violated	Accept
2	B = 1	A–B conflict (same slot)	Reject; try B = 2: ok

Step	Assignment	Check	Result
3	C = 1	Conflicts with A	Reject; C = 2 conflicts with B; C = 3 ok
4	D = 1	A before D requires D > 1	Reject; D = 2: ok (C = 3)
5	E = 1	E–D only; 1 ≠ 2	Accept: complete solution

In a harder instance (for example if only two slots were available) the search would fail at C, backtrack to B, then to A and eventually report that no solution exists. This shows two characteristics of backtracking: it is **complete** (finds a solution if one exists) and its **worst case is exponential** ($O(|D|^E)$). The practical performance depends on three decisions:

- **Variable ordering – Minimum Remaining Values (MRV):** choose the exam with the fewest legal slots/halls left (the "fail-first" principle). Tie-break with the **degree heuristic**: choose the exam involved in most constraints (largest number of shared students), which prunes the most neighbours.
- **Value ordering – Least Constraining Value (LCV):** try first the slot that rules out the fewest options for the neighbouring exams, so that a solution is found early.
- **Intelligent backtracking:** conflict-directed backjumping returns to the variable that actually caused the failure instead of the most recent one; this saves a great amount of repeated work on a big conflict graph.

4(iii) How constraint propagation (forward checking) improves efficiency

Plain backtracking discovers a violation only when it tries to assign a variable whose domain has already been emptied by earlier choices, which can be very late. **Constraint propagation** uses the constraints actively to reduce domains *before* the next assignment so failures are detected early.

Forward checking

When a variable X is assigned a value v, forward checking looks at every unassigned variable Y connected to X by a constraint and deletes from the domain of Y the values that are inconsistent with X = v. If any domain becomes empty the algorithm backtracks immediately.

Step	Assignment	Domains after forward checking (slots)	Comment
0	(none)	A{1,2,3} B{1,2,3} C{1,2,3} D{1,2,3} E{1,2,3}	Initial; after precedence A<D: D{2,3}
1	A = 1	B{2,3} C{2,3} D{2,3} E{1,2,3}	Neighbours of A (B, C) lose slot 1
2	B = 2	C{3} D{2,3} E{1,2,3}	C loses slot 2 because of B
3	C = 3	D{2} E{1,2,3}	D loses slot 3 because of C

Step	Assignment	Domains after forward checking (slots)	Comment
4	D = 2	E{1,3}	E loses slot 2; no domain is empty, so continue
5	E = 1	—	Solution found without any backtracking

Had we chosen B = 3 at step 2 with a different problem where C needs 3, forward checking would have emptied C's domain at once and the algorithm would have backtracked without ever trying C. With plain backtracking the failure would only be noticed when C is selected, after assigning possibly many unrelated exams in between.

Stronger propagation

- **Arc consistency (AC-3).** An arc X→Y is consistent if for every value in D(X) there exists a compatible value in D(Y). AC-3 repeatedly removes values that have no support and propagates the consequences. Maintaining arc consistency (MAC) after each assignment detects failures forward checking misses (for example when two unassigned exams both are left with only the same single slot and conflict).
- **Global constraints.** The AllDiff constraint (exams that mutually conflict need distinct slots) and a "number of exams per slot ≤ number of halls" constraint can be propagated with specialised algorithms (matching-based filtering) that prune much more than pairwise checking.
- **Node consistency.** Before search starts, unary constraints (hall capacity, faculty availability, accommodations) remove impossible values at once, shrinking every domain.

Efficiency benefits

- Early detection of dead ends, which reduces the number of nodes visited by orders of magnitude on tightly constrained instances.
- Reduced domains make MRV much more informative (domain sizes are accurate), so the next choice is better.
- Cheap: forward checking costs O(number of constraints on the assigned variable) per assignment. MAC costs more per node but usually pays off on hard instances.
- Domain wipe-out gives the reason for the failure, which can drive conflict-directed backjumping.

4(iv) Real-world challenges and the best strategy for scalability

Practical challenges

Challenge	Description	Impact
Problem size	Thousands of exams and tens of thousands of students; a dense conflict graph.	Exponential worst case; memory for domains and constraints.

Challenge	Description	Impact
Over-constrained instances	Hard constraints may be impossible to satisfy together (not enough halls for the available slots).	Pure CSP returns "no solution"; need relaxation or negotiation.
Soft preferences	Spread exams, avoid back-to-back exams, fairness; conflicts between preferences.	Becomes a constraint optimisation problem (COP).
Late changes	Course withdrawals, hall unavailability, a faculty member falls ill, new accommodation requests.	Need repair without recomputing a whole timetable and without moving all students' exams.
Heterogeneous halls	Different capacities, split exams across halls, distances between halls, invigilator ratios.	Many more variables and side constraints.
Fairness and acceptability	Stakeholders (students, faculty) must accept the timetable; explanations required.	Needs transparent constraint models and tunable weights.
Data quality	Enrolment data errors lead to missed conflicts.	Invalid timetables discovered late.

Recommended strategy

Recommendation
A **layered CSP strategy**: model all hard rules as a CSP; solve with **backtracking + MRV/degree + LCV + forward checking / MAC (with global AllDiff propagation) and conflict-directed backjumping**; use **problem decomposition** by independent components and **randomised restarts**; improve soft-constraint quality and repair late changes with **local search (min-conflicts, tabu search or simulated annealing)** starting from the feasible timetable.

1. **Pre-processing.** Apply node consistency and AC-3, build the conflict graph, and compute a greedy colouring (e.g. DSATUR) to get a lower and an upper bound for the number of slots required.
2. **Decompose.** Split the conflict graph into connected components or clusters (e.g. by faculty) and solve them independently/in parallel; solve the most constrained (large-enrolment, accommodation) exams first.
3. **Systematic search with strong heuristics.** Backtracking with MRV + degree, LCV and MAC. Use restarts with randomisation and a time limit to avoid being trapped in an early bad decision (heavy-tailed behaviour).
4. **Optimise soft constraints.** From a feasible timetable apply **min-conflicts** local search: pick a conflicted or poor-quality exam, move it to the slot that minimises violations/penalty. Min-conflicts solves even million-variable problems rapidly because the density of solutions in such problems is high.
5. **Incremental repair.** For late changes, keep the existing timetable and fix only the affected exams with local search or by re-running a small CSP on the neighbourhood, with a penalty for moving other exams.
6. **Relaxation for over-constrained cases.** If no solution exists, convert some hard constraints into weighted soft ones (e.g. use an additional slot or an extra hall at a

cost) and solve as a constraint optimisation problem (branch and bound) so that the least-bad timetable is produced with an explanation of which constraints were violated.

```
# Python sketch (forward checking + MRV) for the exam CSP
def solve(domains, conflicts, precedes):
    def consistent(e, s, asg):
        return all(asg.get(o) != s for o in conflicts[e]) and \
               all(asg[a] < s for a in precedes.get(e, []) if a in asg)
    def bt(asg, dom):
        if len(asg) == len(domains): return asg
        e = min((x for x in dom if x not in asg), key=lambda x: len(dom[x]))   # MRV
        for s in sorted(dom[e]):
            if consistent(e, s, asg):
                new = {x: set(v) for x, v in dom.items()}
                new[e] = {s}
                for o in conflicts[e]:                    # forward checking
                    if o not in asg: new[o].discard(s)
                if all(new[x] for x in new):              # no domain wiped out
                    r = bt({**asg, e: s}, new)
                    if r: return r
        return None
    return bt({}, domains)
```

Why this is the best choice. Exact CSP search guarantees that every hard rule (no clashes, capacity, precedence, availability) is respected, which is non-negotiable for exam fairness. Propagation and heuristics push exact search to practical sizes; decomposition and parallelism provide scalability; and local search handles the soft quality criteria and late changes where an exact method would be too slow. Together they scale from a department to a whole university while remaining explainable.

Question 5: Strategic Game AI with Minimax and Alpha-Beta Pruning

5.0 Understanding the scenario

A gaming company needs an AI opponent for a real-time strategy (RTS) game. The AI competes against human players, must answer within strict time limits, must judge complex positions with many possible moves, and faces a decision tree that is far too large to search completely. Minimax and its refinements are the standard answer for two-player, zero-sum, adversarial decision making. A pure RTS is simultaneous and imperfect-information, so in practice the game is **abstracted into discrete decision turns** (for example every 0.5 s) over a simplified state (resources, unit groups, positions, map control); minimax then searches on this abstraction. Where the original game has hidden information (fog of war) the AI works on its belief state or samples possible worlds.

5(i) How the Minimax algorithm models this problem

Minimax models the game as an **adversarial search problem** with two players: **MAX** (the AI) who tries to maximise the evaluation, and **MIN** (the human) who is assumed to play the best replies to minimise it. This assumption makes the AI robust: it chooses the move that gives the best outcome against the strongest opposition.

Game element	RTS formulation
Initial state S_0	Current game snapshot: resources, buildings, unit counts and positions, map control, technology level.
PLAYER(s)	Whose turn it is (MAX = AI, MIN = human) in the discretised game.
ACTIONS(s)	Legal macro-actions: build unit or structure, research, move/attack with a group, expand to a new base, defend, retreat.
RESULT(s, a)	Next state after the action and a fixed time step (using a forward model/simulator).
TERMINAL-TEST(s)	Base destroyed, victory/defeat condition, or the search depth/time limit is reached (cut-off).
UTILITY(s) / EVAL(s)	$+\infty$ for win, $-\infty$ for loss; otherwise a heuristic evaluation (see 5(iv)).

```
function MINIMAX-DECISION(state): return argmax over a in ACTIONS(state) of MIN-VALUE(RESULT(state, a))

function MAX-VALUE(s):                          function MIN-VALUE(s):
    if TERMINAL(s): return UTILITY(s)               if TERMINAL(s): return UTILITY(s)
    v <- -infinity                                  v <- +infinity
    for a in ACTIONS(s):                            for a in ACTIONS(s):
        v <- max(v, MIN-VALUE(RESULT(s,a)))             v <- min(v, MAX-VALUE(RESULT(s,a)))
    return v                                        return v
```

Properties. For a finite game tree minimax is **complete** and, against an optimal opponent, **optimal**. Time complexity is $\mathbf{O(b^m)}$ and space complexity O(b·m) (depth-first), where b is the branching factor and m the maximum depth.

Worked example (depth-2 tree)

MAX moves first and has three actions a_1, a_2, a_3; each leads to a MIN node with three replies. Leaf utilities are shown below.

MAX move	MIN replies → leaf values	MIN picks (minimum)
a_1	3, 12, 8	3
a_2	2, 4, 6	2
a_3	14, 5, 2	2

MAX takes the maximum of the MIN values {3, 2, 2} = **3**, so the AI plays a_1. Even though a_3 contains the attractive leaf 14, a rational opponent would never allow it, and minimax correctly avoids the trap. In an RTS this corresponds to rejecting an aggressive rush that looks great if the human does not defend but loses badly if the human counter-attacks.

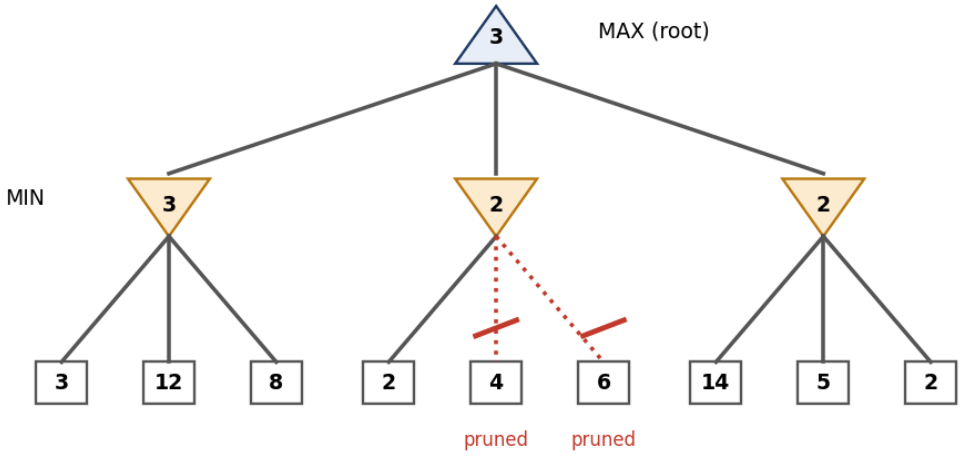


Figure 6: Alpha-Beta pruning on the depth-2 example; leaves 4 and 6 are pruned

5(ii) Computational challenges of large game trees

- **Exponential growth.** With branching factor b and depth d the tree has about b^d leaves. Chess has b ≈ 35: depth 10 gives $35^{10} \approx 2.8 \times 10^{15}$ positions. An RTS is far worse: the number of possible unit commands per step may be 10^{30} or more (every unit may take several actions), so even depth 2 is impossible.
- **Time limit.** The AI may have 10–50 ms per decision. If a machine evaluates 10^6 nodes per second, only about 10^4 nodes can be examined, which allows a depth of only 2–3 plies at b = 30.
- **Evaluation cost.** Every leaf needs the forward model and the evaluation function; a rich evaluation may cost microseconds to milliseconds.
- **Horizon effect.** A fixed depth cut-off may hide a disaster that happens just beyond it (or postpone the unavoidable by delaying moves), so the AI makes bad choices that look good at the horizon.

- **Repeated states.** Different move orders lead to the same position (transpositions), causing the same subtree to be searched many times.
- **Uncertainty and simultaneity.** Fog of war and simultaneous moves violate the perfect-information, turn-based assumptions of minimax; the AI must sample or estimate hidden information.
- **Memory.** Storing the whole tree is impossible; depth-first search with limited memory and a transposition table is needed.

Depth d	b = 10	b = 35	b = 100
2	100	1,225	10,000
4	10,000	1.5×10^6	10^8
6	10^6	1.8×10^9	10^{12}
10	10^{10}	2.8×10^{15}	10^{20}

5(iii) How Alpha-Beta pruning improves efficiency (with detailed reasoning)

Alpha-beta pruning returns **exactly the same decision as minimax** but skips (prunes) branches that cannot influence the final choice. It maintains two bounds during the depth-first traversal:

- **α (alpha)** = the best (highest) value found so far for MAX along the path to the current node: a guaranteed lower bound for MAX.
- **β (beta)** = the best (lowest) value found so far for MIN along the path: a guaranteed upper bound for MIN.

```
function ALPHA-BETA-SEARCH(state): v <- MAX-VALUE(state, -inf, +inf); return the action with value v

function MAX-VALUE(s, alpha, beta):         function MIN-VALUE(s, alpha, beta):
    if TERMINAL(s): return UTILITY(s)           if TERMINAL(s): return UTILITY(s)
    v <- -inf                                   v <- +inf
    for a in ACTIONS(s):                        for a in ACTIONS(s):
        v <- max(v, MIN-VALUE(RESULT(s,a),          v <- min(v, MAX-VALUE(RESULT(s,a),
                              alpha, beta))                               alpha, beta))
        if v >= beta: return v   # beta cut-off      if v <= alpha: return v  # alpha cut-off
        alpha <- max(alpha, v)                      beta <- min(beta, v)
    return v                                    return v
```

Reasoning for why pruning is safe

Suppose MAX is at the root and has already found a move worth 3 (so α = 3). Now MAX examines another move a_2 whose MIN node sees its first reply with value 2. MIN will choose a value ≤ 2 at that node, whatever the other replies are. Since MAX already has a guaranteed 3, MAX would never choose a_2 (it can give at most 2). The remaining replies of a_2 need not be evaluated: they cannot raise the node's value above 2 (MIN would simply not choose them if they are higher, and if they are lower it only makes a_2 worse). This is an **alpha cut-off**. A symmetric **beta cut-off** happens at a MAX node when its value reaches or exceeds a bound that MIN above already

can avoid.

Trace on the depth-2 example

Step	Node visited	α	β	Action
1	Root MAX: α = −∞, β = +∞	−∞	+∞	Go to a_1 (MIN node)
2	a_1 leaf 3	−∞	3	β = 3
3	a_1 leaf 12, leaf 8	−∞	3	No change; a_1 = 3, so root α = 3
4	a_2 leaf 2	3	2	v = 2 ≤ α = 3 → **prune** leaves 4 and 6 (not examined)
5	a_3 leaf 14	3	14	continue; v = 14
6	a_3 leaf 5	3	5	continue; v = 5
7	a_3 leaf 2	3	2	v = 2 ≤ 3; a_3 = 2
8	Root			Value = max(3, 2, 2) = 3; best move a_1

Two of nine leaves (22%) were never evaluated, and the answer (3, move a_1) is identical to minimax. In large trees the savings are enormous.

Efficiency analysis

- **Worst case** (moves ordered worst-first): no pruning, so $O(b^m)$, the same as minimax.
- **Best case** (perfect move ordering): $O(b^{m/2})$. The AI can look *twice as deep* in the same time, or, equivalently, the effective branching factor falls from b to about √b (35 → about 6).
- **Random ordering:** about $O(b^{3m/4})$.
- **Move ordering is the key.** Examine promising moves first: captures/attacks, killer moves (moves that caused cut-offs at the same depth in sibling nodes), history heuristic (moves that were good earlier), and the best move from the previous iteration (iterative deepening).
- Additional improvements that combine well: **transposition tables** (cache values of positions already searched), **aspiration windows** (start with a narrow (α, β) window around the expected value), **principal variation search**, and **quiescence search** (extend the search in volatile positions such as ongoing battles to reduce the horizon effect).

Effect	Minimax	Alpha-Beta (best order)
Nodes at depth 6, b = 30	≈ 7.3×10^8	≈ 2.7×10^4 (about 30^3)
Result	Optimal move	Same optimal move
Depth achievable in the same time	d	up to 2d

5(iv) Design of an evaluation function and justification of the factors

Because the AI cannot search to the end of the game, it applies a **heuristic evaluation function EVAL(s)** at the cut-off depth, which estimates the expected utility of the state from MAX's viewpoint. A good function must be **fast** (it is called at every leaf), **correlated with winning chances**, and must **agree with the true utility at terminal states** (win = $+\infty$).

A standard approach is a **weighted linear function**: $\mathbf{EVAL(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)}$, where each f_i is a normalised feature computed as (AI value − opponent value) and w_i is its weight.

Feature f_i	**Definition**	**Justification**	**Initial weight**
f_1 Military strength	Σ (unit cost × health) of AI units − same for the opponent, adjusted by counter relationships (e.g. archers beat infantry)	Winning battles is the most direct way to win; counters make raw counts misleading	0.30
f_2 Economy	Resource income rate + stored resources + number of workers	Economy converts into future army; strong economy predicts a long-term advantage	0.20
f_3 Map control / territory	Fraction of key points, expansions and vision controlled	Controls resources and lets the AI attack or retreat on its own terms	0.15
f_4 Technology / upgrades	Tech level and research completed	Gives quality advantages and unlocks stronger units	0.10
f_5 Base safety (defence)	Defensive structures, distance of enemy forces to base, army at home	Avoids being destroyed by a surprise attack, which would make all else irrelevant	0.15
f_6 Tempo / production capacity	Number of production buildings and queue utilisation	Ability to reinforce quickly decides prolonged fights	0.05
f_7 Information / scouting	Fraction of enemy base and army known	Reduces the uncertainty behind fog of war	0.05

Design principles and justification

- **Zero-sum symmetry:** each feature is (AI − opponent), so EVAL(s) = −EVAL(s with roles swapped), consistent with minimax.
- **Normalisation:** every feature is scaled to [−1, 1], so that weights express importance rather than units.
- **Non-linear terms where needed:** an economy advantage matters little if the base is about to fall; interaction can be captured with product terms or phase-dependent weights (e.g. economy weighted more in the early game, military more in the late game).
- **Terminal states:** EVAL = $+\infty$ (enemy base destroyed) and $-\infty$ (own base destroyed), and a heavy penalty for states in which the base is under immediate threat.

- **Speed:** features are updated incrementally rather than recomputed from scratch, since the function is called millions of times.
- **Tuning:** weights are tuned by self-play, by regression on game logs (predicting win/loss from features), or by evolutionary search; a logistic transform maps EVAL to a win probability.
- **Quiescence:** apply EVAL only to *quiet* states (no battle in progress), as evaluation of a mid-battle snapshot is unreliable.

Example calculation. At the cut-off a position gives (normalised) $f_1 = +0.4$, $f_2 = -0.1$, $f_3 = +0.5$, $f_4 = 0$, $f_5 = +0.2$, $f_6 = 0.1$, $f_7 = 0$. EVAL = $0.30(0.4) + 0.20(-0.1) + 0.15(0.5) + 0.10(0) + 0.15(0.2) + 0.05(0.1) + 0 = 0.12 - 0.02 + 0.075 + 0 + 0.03 + 0.005 =$ **0.21**, a moderately favourable position for the AI.

5(v) Strategy for balancing optimality and real-time performance

Recommendation
Use **iterative-deepening alpha-beta search with a strict time budget**, strong move ordering, a transposition table, quiescence search and a tuned evaluation function, operating on an **abstracted action space** (macro-actions, groups of units). Where the branching factor is still too high, **blend with Monte Carlo Tree Search (MCTS)** or **portfolio / script-based search**.

1. **Iterative deepening with time management.** Search depth 1, then 2, 3, …, always keeping the best move of the last *completed* iteration. When the time budget (for example 30 ms) expires, the AI plays that move. This gives an **anytime** algorithm: it always has an answer and uses all the available time. The shallow iterations are cheap and their best move is searched first in the next iteration (excellent move ordering).
2. **Reduce the branching factor by abstraction.** Group units into squads and use macro-actions (attack with squad, expand, build army); consider only a limited number of promising candidate moves per node (selective or beam search).
3. **Move ordering and pruning.** Killer moves, history heuristic, transposition-table move first. Optional **forward pruning** (null-move pruning, late-move reductions) trades small risk of error for large depth; reserve these for non-critical positions.
4. **Transposition table.** Store values and best moves of searched states with a hash key so identical positions reached by different move orders are not searched twice.
5. **Quiescence / selective extensions.** Extend the search where battles or threats occur and shorten it in quiet positions.
6. **Adaptive depth and difficulty.** Choose depth from remaining time and the current branching factor; for easier difficulty levels reduce depth or add noise to the evaluation so that the AI remains beatable.
7. **Handle real-time and imperfect information.** Run the search in a separate thread on a periodic schedule (e.g. every 0.5 s), execute the current plan between

searches, and sample the hidden enemy state (determinisation) or use MCTS rollouts for imperfect information.

8. **Learning-based improvements.** Tune evaluation weights from self-play; use a learned value or policy model to order moves and evaluate states more accurately (as in modern game AI).

Design choice	Gain	Cost / risk
Alpha-beta pruning	Same result as minimax, up to double depth	None in quality; needs good ordering
Iterative deepening	Anytime, reliable time control, better ordering	Small repeated work (about 1/b overhead)
Transposition table	Avoids repeated work	Memory; hash collisions
Depth limit + evaluation	Makes search feasible	Horizon effect; sub-optimal play
Selective / forward pruning	Much deeper search	May prune the best move
MCTS hybrid	Works with huge b, no handcrafted evaluation required	Randomness; less exact tactics

Conclusion. Pure minimax guarantees the best play but is impossible in real time; alpha-beta pruning keeps the same decision while greatly reducing computation. Combined with iterative deepening, move ordering, an informed evaluation function and abstraction of the action space, it gives a strong opponent that always responds within the time limit, accepting a bounded loss of optimality at the horizon in exchange for practical, real-time performance.